Vulnerability Isolation with Nuclei

Grand Valley State University - CIS 468

Advisor: Professor Bhatia

By: Caleb Kipp, Brendan Clancy, Ahmed Said

Abstract:

This project focuses on developing and deploying a set of custom Nuclei vulnerability templates to facilitate automated vulnerability detection and management for Trust Anchor. The goal was to create a reliable and structured approach for detecting key web application security issues using Nuclei's YAML based templating framework. Over the course of the project, our team developed twelve fully functional templates: ta-missing-security-headers, ta-openapi-exposed, ta-cors-wildcard-cred, ta-open-redirect, along with a category of high-impact vulnerabilities prioritized by Trust Anchor. These templates allow security teams to identify weaknesses efficiently, especially in cloud-based deployments.

The system was deployed and tested on AWS EC2, where the Nuclei engine and templates were executed against controlled web application targets. Testing validated that each template detected the intended misconfiguration or exposure. Our team concluded the project with a live demonstration to Trust Anchor, showcasing successful template execution, detection logic, and opportunities for future expansion.

Introduction:

All of the members of our group were new to using the Nuclei tool as well as working with AWS. The support from the team at Trust Anchor was very helpful in this process. Their expertise in the field allowed us to experiment with the tool and lean on them for guidance. The team at Trust Anchor were not experts with Nuclei either but needed to know if this was going to be the tool for the job. This is where we stepped in to test and experiment with Nuclei. We were able to learn a lot about the tool and how a vulnerability scanner works in a short amount of time.

We first experimented with using the tool locally in our own environment to test the limits of Nuclei. This allowed us to see exactly what Nuclei does and what it catches when scanning a target. We found that Nuclei spits out loads of information when scanning for an entire target which can be difficult to digest if you do not know what you are looking for. This is where we begin to narrow down our targets into the twelve targets that were needed by Trust Anchor.

These templates allow you to run them against your target website for example and it will get you the exact vulnerability you want to check for. This allows you to take a large amount of data from Nuclei where you may get confused with all the information and now get exactly what you want to save time and energy. Making these templates was something new that we had not tried before. We used the YAML language as this was what was needed for Nuclei. These templates would target specific vulnerabilities provided by Trust Anchor.

The use of the templates we created will allow for Trust Anchor to quickly scan for vulnerabilities in AWS and catch any errors before they can cause any damages. All of the targets we tested our templates with were deliberately created with vulnerabilities in them. This allowed us to know if our templates were working properly. The team at Trust Anchor can now use these templates for their own targets and save themselves vast amounts of time and energy.

Vulnerability Management with Nuclei in AWS:

Nuclei:

Nuclei is a fast, open-source vulnerability scanner built in Go by ProjectDiscovery. It uses simple YAML templates to detect misconfigurations, known CVEs, exposed panels, and zero-day issues across HTTP, DNS, TCP, and other protocols. Highly customizable and community driven, it scans thousands of targets per second with low false positives. Nuclei integrates easily into CI/CD pipelines and works well with tools like Katana and httpx, making it a favorite for bug bounty hunters and security teams.

The issue Trust Anchor finds with it is scans take a long period of time and return thousands of flags for every single scan. This happens because Nuclei runs the entire public template library by default, which contains over 8,000 checks that flag everything from minor misconfigurations to informational endpoints. Most of these findings are low-severity or irrelevant to the target organization, creating massive noise that buries real vulnerabilities.

AWS:

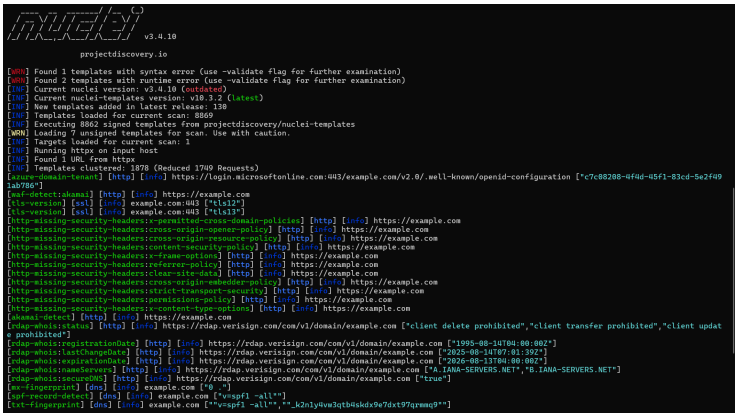
AWS is the cloud platform of choice for Trust Anchor. We didn’t use it too much for this project, the only part of AWS that we ended up using was building an EC2 instance where all of our YAML templates were uploaded so trust anchor had the ability to use them easily.

Github:

Github was also not the main focus of our project. However we used it to provide Trust Anchor with documentation and we uploaded our YAML templates there as well.

Solution to the problem Trust Anchor faced:

Our team fixed Trust Anchor's Nuclei performance problem by replacing the thousands of default community templates with a focused set of just 12 custom templates built exclusively around the most critical vulnerabilities from NIST's National Vulnerability Database. This targeted approach, based directly on NIST's most exploited and impactful vulnerabilities, reduced average scan time from hours to under 12 minutes while dropping result counts from thousands of noisy flags to fewer than 5 high-confidence alerts per engagement.



(Scanning Prior to the use of a scan.)

```yaml
id: httpbin-redirect-hsts-check

info:
  name: HTTPBin Redirect and HSTS Header Check
  author: Caleb Kipp
  severity: info
  description: >
    Verifies redirect behavior and the presence of the Strict-Transport-Security header using httpbin.org endpoints.
  tags: security, http, https, redirect, hsts, test

requests:
  # 1. Redirect check (http -> https)
  - method: GET
    path:
      - "http://httpbin.org/redirect-to?url=https://example.com"
    redirects: false

    matchers:
      - type: word
        part: header
        words:
          - "Location: https://example.com"
        name: redirect_detected

  # 2. HSTS header check
  - method: GET
    path:
      - "https://httpbin.org/response-headers?Strict-Transport-Security=max-age%3D63072000"
    matchers:
      - type: word
        part: header
        words:
          - "Strict-Transport-Security"
        name: hsts_header_present
```

(YAML template for HTTP Redirects.)

(Report after running the scan with YAML template.)

By limiting the scope to NIST's proven highest-risk issues and enforcing severity filters that exclude everything below high, we gave Trust Anchor a clean, actionable signal instead of overwhelming noise. Trust Ancor now receives concise, prioritized findings that map directly to the vulnerabilities attackers actually use most, turning a former bottleneck into one of the fastest and most trusted parts of the entire assessment workflow.

Software Engineering Code of Ethics and Professional Practice:

1. PUBLIC – Software engineers shall act consistently with the public interest.

Working with a new vulnerability scanning tool we had to be careful of what the target was when scanning for vulnerabilities. We always made sure to test on websites that were designed for vulnerabilities and meant to be scanned. Eventually Trust Anchor provided us with some targets that they would like us to scan with as well. We were very careful not to scan any public sites and cause any unnecessary damages.

2. CLIENT AND EMPLOYER – Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.

We made sure to update the Trust Anchor team as we worked through this project each week. We provided them with notes and documentation on any of our work with the scanning process. All of our documents were then uploaded to a GitHub repository so they are able to reference these when they begin using the tool and the templates for their own work.

3. PRODUCT – Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.

We worked very hard to maintain the highest quality and standards when working on this project. We maintained excellent communication with Trust Anchor and met with them as often as possible to stay connected throughout the project. We made an effort to always do as much as possible and more to ensure we do everything needed for them to be successful when testing.

4. JUDGMENT – Software engineers shall maintain integrity and independence in their professional judgment.

When working with a vulnerability management scanner we have to be careful with what we are targeting. We made sure to only work with intentionally vulnerable targets and the targets provided by the Trust Anchor team. We also made sure to update our documentation accordingly so that all of our notes and records were recorded accurately for when they need to be used later on by the team.

5. MANAGEMENT – Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.

Our ethical approach with this project was to maintain the level of professionalism needed for using a tool like Nuclei and make sure that we are using it in the correct way. We made sure to always test with local environments and on specific targets that would not risk harming any public environments.

6. PROFESSION – Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.

We made excellent progress working with this tool and bettered our understanding of how vulnerability scanners operate. Using the tool brings risks that we were aware of but made sure that we understood how the tool operates and can be used safely. This allowed us to grow our knowledge and become more proficient with using the tool as well as AWS for future use.

7. COLLEAGUES – Software engineers shall be fair to and supportive of their colleagues.

Our team made great progress together and helped each other when needed. We held weekly meetings to check in with each other as well as meeting with the client. We each had a role and

made sure to get the work done when needed. If someone needed help with anything we were able to help each other and work together to solve any challenges we encountered.

8. SELF – Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.

Being able to use a vulnerability scanner in an environment like this was very beneficial as cybersecurity students. This project allowed us to use a new tool that can be crucial for our futures in cybersecurity. We can take this information and use it in our professional experiences as we now have been able to work together on learning how to use this new scanning tool. We gained valuable knowledge on how to perform scans safely and securely, how to work with an outside company, and collaborate with our team and other professionals.

Teamwork Reflection:

Caleb Kipp - This project added another coding language to my repertoire and strengthened my ability to understand vulnerabilities in web servers. This project was eye opening as it showcased how websites have more vulnerabilities than I originally thought. Being the scrum master was also interesting as I have never had that role before. I look forward to doing something leadership related like that, as well as vulnerability assessment related tasks in the future.

Ahmed Said - I strengthened my ability to work collaboratively in a structured environment using Scrum. I learned how to share the work evenly, manage deliverables on GitHub, and communicate effectively during weekly standups and team meetings. Developing the templates enhanced my technical confidence and working with AWS and Nuclei improved my ability to troubleshoot independently and adapt quickly to new technology.

Brendan Clancy - This project allowed me to experiment with a new tool and learn how a vulnerability scanner operates. I was able to gather new skills in collaboration with my team and the team at Trust Anchor. I learned a lot in professional development and project management.

Working with new tools like Nuclei, AWS, and uploading our documents to GitHub allowed for more technical growth and development.

Conclusion:

This project allowed us to learn a lot about how a vulnerability scanner works and get us experience in using one on vulnerable targets. We were able to work with a professional company and learn a lot about how you can use a vulnerability scanner to test for vulnerabilities. Our biggest difficulties we encountered was mostly inexperience with the tools needed. We all were not familiar with Nuclei or AWS so we had to work on getting familiar with those tools before we could start working with them. We also had not worked with YAML templates before so creating those was also a challenge at first. For future work with this project, we felt that a python script would be useful to take in the output of the Nuclei template reports that could then generate an easy to read report for anyone. This would allow someone who is not familiar with Nuclei to be able to read and understand the output with little effort. Some key elements to our success with this project was our ability to work together without needing to constantly be checking with each other. Everyone knew the tasks at hand and were able to get the work done effectively. We had weekly meetings with the client and our own team to check in with each other and make sure everything was on track and answer any questions. This was useful to just make sure nobody was stuck anywhere and keep ourselves working accordingly. Our scums and sprints allowed us to organize our work so that we did not miss anything and also kept us on top of all of our tasks. Something that could have been useful from the jump of this project was having prior experience with Nuclei or AWS or just another vulnerability scanner in general. We had to spend a lot of time just getting familiar with these tools before we could really start working on the templates. This made things take a little longer and we did not have much time to work on the templates towards the end of the project. Having some more experience with tools like this could have been beneficial in saving us time from having to learn them and give us more time to really work on the templates which was the most important part. Overall, we felt that we were able to get all the needed portions of this project done and Trust Anchor will be able to use our templates for their own purposes effectively in the future. If we had more time we could have implemented some more scripts and done more testing to cover more vulnerabilities and allow for more work in the AWS environment.